

Staj Projesi Teknik Dökümanı

eBPF ile Gerçek Zamanlı Uygulama Performans İzleme Dashboard

Proje Türü	eBPF Araştırma ve Web Backend Geliştirme
Süre	6-8 Hafta

1. Proje Özeti

Bu proje, eBPF teknolojisini basit seviyede öğrenmeyi ve bunu gerçek bir web uygulamasına entegre etmeyi hedeflemektedir. Öğrenci, hazır eBPF araçlarını (bpfftrace, BCC tools) kullanarak sistem ve uygulama metrikleri toplayacak, ardından bu verileri görselleştiren bir web dashboard geliştirecektir.

Çıktı: Kullanıcıların sistem performansını (CPU, network, disk, process metrikleri) gerçek zamanlı izleyebileceği, eBPF ile toplanan verileri görselleştiren bir web dashboard.

2. Teknik Kapsam

2.1 eBPF Bileşenleri (Hazır Araçlar)

bpfftrace: Yüksek seviye tracing dili. Basit one-liner'larla sistem eventi yakalama.

Örnek: `bpfftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'`

BCC Tools: Python/C tabanlı hazır eBPF programları (execsnoop, tcpconnect, biolateness).

Örnek: `/usr/share/bcc/tools/execsnoop` (yeni process'leri yakalar)

eBPF Exporter (opsiyonel): Prometheus-compatible metrics exporter. Daha ileri seviye için.

2.2 Backend Bileşenleri

Web Framework: Flask veya FastAPI (Python). Basit ve hızlı geliştirme.

Data Collection Service: Background worker'lar bpfftrace/BCC araçlarını periyodik çalıştırır.

Data Storage: SQLite (basit) veya Redis (time-series için).

REST API: Toplanan metrikleri frontend'e sunma.

Dashboard: Basit HTML/JS veya React ile görselleştirme (Chart.js kullanılabilir).

2.3 Toplanacak Metrikler (Örnekler)

- Sistem çağırısı istatistikleri (hangi syscall kaç kez çağrıldı)
- Yeni başlatılan process'ler ve komutları
- TCP bağlantı olayları (hangi IP'lere bağlanıldı)
- Disk I/O latency dağılımı
- En çok CPU kullanan fonksiyonlar (profiling)
- Network paket istatistikleri

3. Proje Aşamaları ve Yol Haritası

Faz 1: eBPF Araştırma ve Deneme (1.5 Hafta)

Hedefler:

- eBPF konseptini kavramak
- bpftrace ve BCC tools ile deneyler yapmak
- Araçların çıktılarını anlamak

Yapılacaklar:

Ubuntu 22.04+ VM kurulumu

bpftrace ve bcc-tools kurulumu (apt install)

```
sudo apt install bpfcc-tools linux-headers-$(uname -r) bpftrace
```

10-15 farklı BCC tool'u manuel olarak çalıştırıp çıktılarını incelemek:

- execsnoop: Yeni process'leri izle
- opensnoop: Açılan dosyaları izle
- tcpconnect: TCP bağlantıları
- biolatency: Disk I/O latency
- cpudist: CPU kullanım dağılımı

bpftrace one-liner'ları denemek (ebpf.io/what-is-ebpf tutorial)

Araçların çıktısını dosyaya kaydetmek ve formatını analiz etmek

Dokümantasyon:

eBPF nedir, nasıl çalışır? (1-2 sayfa açıklama)

Denenen her tool için: ne yapar, çıktısı nasıl, ne öğrenildi

Örnek çıktılar (screenshots veya text)

Faz 2: Backend API Geliştirme (2 Hafta)

Hedefler:

- Flask/FastAPI ile REST API oluşturmak
- eBPF araçlarını Python'dan çalıştırıp çıktılarını parse etmek
- Verileri depolamak ve sorgulamak

Yapılacaklar:

Flask veya FastAPI projesi oluşturma

subprocess modülü ile eBPF araçlarını çalıştırma

```
result = subprocess.run(['execsnoop-bpfcc', '-T'], capture_output=True, timeout=5)
```

Araç çıktılarını parse etme (regex, split, json parse)

SQLite veya Redis'e metrik kaydetme

Background task scheduler (APScheduler veya Celery)

API endpoint'leri tasarlama:

- GET /api/metrics/syscalls (sistem çağrısı stats)
- GET /api/metrics/processes (yeni process'ler)
- GET /api/metrics/network (network aktivite)
- GET /api/metrics/disk (disk latency)

Teknik Detaylar:

Parser Örneği: *execsnoop çıktısı genelde:*

TIME	PID	COMM	ARGS
12:34:56	1234	bash	/usr/bin/ls -la

Bu satırları split() ile parse edip JSON'a çevirme

Çıktılar:

Çalışan REST API servisi

En az 3-4 farklı metrik toplayan endpoint

Veri depolama mekanizması

Faz 3: Dashboard ve Görselleştirme (2 Hafta)

Hedefler:

- Web arayüzü geliştirmek
- Metrikleri grafikler ile göstermek
- Gerçek zamanlı veri güncellemesi (polling veya WebSocket)

Yapılacaklar:

HTML/CSS/JavaScript ile basit dashboard

Chart.js veya Plotly.js ile grafik oluşturma

API'den veri çekme (fetch() ile)

Gösterilecek widget'lar:

- Sistem çağrısı bar chart (en çok çağrılan 10 syscall)
- Son process'ler tablosu
- Disk I/O latency histogram
- Network bağlantıları timeline
- CPU profiling flame graph (opsiyonel)

Auto-refresh mekanizması (her 5 saniyede bir güncelleme)

Responsive tasarım (Bootstrap veya Tailwind)

Çıktılar:

Fully functional web dashboard

En az 4-5 farklı metrik görselleştirmesi

Gerçek zamanlı veri güncellemesi

Faz 4: İyileştirmeler ve Dokümantasyon (1.5 Hafta)

Hedefler:

- Kod kalitesini arttırmak
- Hata yönetimi eklemek
- Kapsamlı dokümantasyon yazmak

Yapılacaklar:

Error handling (tool timeout, parse error, permission denied)

Logging sistemi (Python logging modülü)

Unit test'ler (parser fonksiyonları için)

Docker container'ı oluşturma (Dockerfile + docker-compose.yml)

README.md yazma:

- Proje tanımı
- Kurulum (dependencies, eBPF tools setup)
- Çalıştırma talimatları
- API endpoint'leri dokümantasyonu
- Screenshot'lar

Architecture diagram (sistem bileşenleri, veri akışı)

Teknik rapor (eBPF öğrendikleri, karşılaşılan zorluklar, çözümler)

4. Teknik Detaylar ve İpuçları

4.1 eBPF Araçlarını Python'dan Çalıştırma

```
import subprocess
```

```
import json
```

```
def collect_execsnoop_data(duration=5):
```

```
    result = subprocess.run(
```

```
        ['sudo', 'execsnoop-bpfcc', '-T'],
```

```
        capture_output=True,
```

```
        timeout=duration,
```

```
        text=True
```

```
    )
```

```
    return parse_execsnoop_output(result.stdout)
```

4.2 Parsing Stratejisi

Her eBPF aracının farklı çıktı formatı vardır. Öğrenci, her aracın çıktısını inceleyip uygun parser yazmalıdır.

- Satır satır okuma (splitlines())
- Header satırını atlama
- Regex veya split() ile kolonları ayırma
- Dictionary veya dataclass'a çevirme

4.3 Önerilen Teknoloji Stack

Bileşen	Teknoloji
Backend Framework	FastAPI (önerilen) veya Flask
eBPF Tools	bpfftrace, BCC tools (apt install bpfcc-tools)
Database	SQLite (basit) veya Redis
Background Tasks	APScheduler (basit) veya Celery
Frontend	HTML/JS/Chart.js veya React (opsiyonel)
Deployment	Docker + Docker Compose

5. Beklenen Çıktılar

5.1 Teknik Çıktılar

1. Çalışan web dashboard uygulaması
2. REST API backend (FastAPI/Flask)
3. eBPF araçlarını çalıştıran collector servisler
4. Parser fonksiyonları (her metrik tipi için)
5. Veri depolama ve sorgulama sistemi

6. En az 4-5 farklı metrik görselleştirme

7. Docker deployment setup

5.2 Dokümantasyon Çıktıları

1. Kapsamlı README.md

2. eBPF Araştırma Raporu (5-7 sayfa)

- eBPF nedir, nasıl çalışır

- Kullanım alanları ve avantajları

- Denenen araçlar ve gözlemler

3. API Dokümantasyonu (endpoint'ler, request/response örnekleri)

4. Architecture Diagram

5. Dashboard screenshot'ları

6. Karşılaşılan problemler ve çözümler belgesi

6. Öğrenme Hedefleri

eBPF Bilgisi:

- eBPF'nin ne olduğunu ve nasıl çalıştığını anlamak
- Linux kernel monitoring ve tracing konseptleri
- Hazır eBPF araçlarını kullanabilmek

Backend Development:

- REST API tasarımı ve implementasyonu
- Subprocess ve external tool entegrasyonu
- Text parsing ve data transformation
- Background task scheduling

Full-Stack Skills:

- Frontend-backend entegrasyonu
- Veri görselleştirme (charts, graphs)
- Real-time data update mekanizmaları

7. Önemli Notlar

7.1 eBPF Öğrenme Yaklaşımı

Bu proje, sıfırdan eBPF C kodu yazmayı gerektirmez. Bunun yerine:

- ✓ Hazır eBPF araçlarını kullanmayı öğrenin
- ✓ Her aracın ne yaptığını, nasıl çalıştığını anlayın
- ✓ BCC GitHub repo'sundaki örnek kodları okuyun
- ✓ İlgiliyseniz, basit bir bpftrace script'i yazabilirsiniz (bonus)

7.2 Başlarken İpuçları

- İlk hafta sadece eBPF araçlarını terminal'den çalıştırın
- Her aracın çıktısını text dosyasına kaydedin
- Basit bir araç ile başlayın (execsnoop)
- Parser'ı önce Jupyter notebook'ta test edin
- Tek bir metrik ile end-to-end pipeline kurun, sonra genişletin

8. Sonuç

Bu proje, eBPF gibi ileri seviye bir teknolojiye giriş yapmayı, hazır araçları kullanarak pratik deneyim kazanmayı ve bunu gerçek bir web uygulamasına entegre etmeyi sağlar. Sıfırdan kernel programlama yapmak yerine, mevcut ekosistemi anlamaya ve kullanmaya odaklanır.

Proje sonunda öğrenci, hem modern Linux monitoring teknolojilerini öğrenmiş, hem de full-stack bir uygulama geliştirme deneyimi kazanmış olacaktır. eBPF alanında edinilen bilgi, cloud-native teknolojiler ve observability alanlarında kariyer için sağlam bir temel oluşturacaktır.

Başarılar dileriz!